DSCI 552 Homework 6

Tree-Based Methods and ISLR Exercises | Reproducible full-data analysis | Random seed 552

1(a). Data loading and validation

The supplied APS CSV files contain a license/preamble before the header. The loader searches for the line starting with class,; in both files the header is line 21 (20 preceding lines). The literal na is parsed as missing. Labels are mapped pos = 1 and neg = 0. All 170 predictors are numeric float64 after parsing, and train/test names and order match exactly. The class column is excluded from every feature matrix.

Official split	Rows	Predictors	neg (0)	pos (1)	Missing cells
Training	60,000	170	59,000	1,000	850,015
Test	16,000	170	15,625	375	228,680

The local description identifies anonymized counters and histogram bins from heavy trucks. A positive label represents a failure of the specific APS component; negatives concern other components. Raw data and the original ZIP/PDF have been retained. File hashes and parsing details are in results/data_audit.json.

1(b)(i). Missing-value methods and choice

Common approaches include complete-case deletion, constant/mean/median/mode imputation, nearest-neighbor imputation, and iterative conditional models; multiple imputation can reflect uncertainty. Median imputation is used here because it is inexpensive, preserves all observations and features, and is less sensitive to extreme counters than a mean [2]. It does not preserve joint distributions or model missingness mechanisms. Missing indicators and alternative imputers are outside this fixed comparison.

Footnote 1 explicitly calls for using test statistics to fill test missing values. Main results therefore use training medians for training and separately computed, unlabeled test medians for test. This differs from the usual deployment workflow, where the fitted training imputer is retained and only transforms future observations. No train/test statistics are pooled and test labels are never used for preprocessing. Page 12 reports the train-only alternative.

Inside cross-validation, each imputer is fitted only on that fold's training rows and then transforms validation rows. No validation statistics, labels, synthetic examples, or official test data enter fitting or selection. The footnote convention is applied only to the final official-test evaluation.

1(b)(ii). Coefficients of variation

All descriptive calculations use the 60,000 training rows after training-median imputation. Coefficients are calculated on the original scale, before any standardization or plotting transform. The sample standard deviation uses ddof = 1.

$$\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}, \qquad s_j = \sqrt{\frac{\sum_i (x_{ij} - \bar{x}_j)^2}{n-1}}, \qquad CV_j = \frac{s_j}{\bar{x}_j}$$

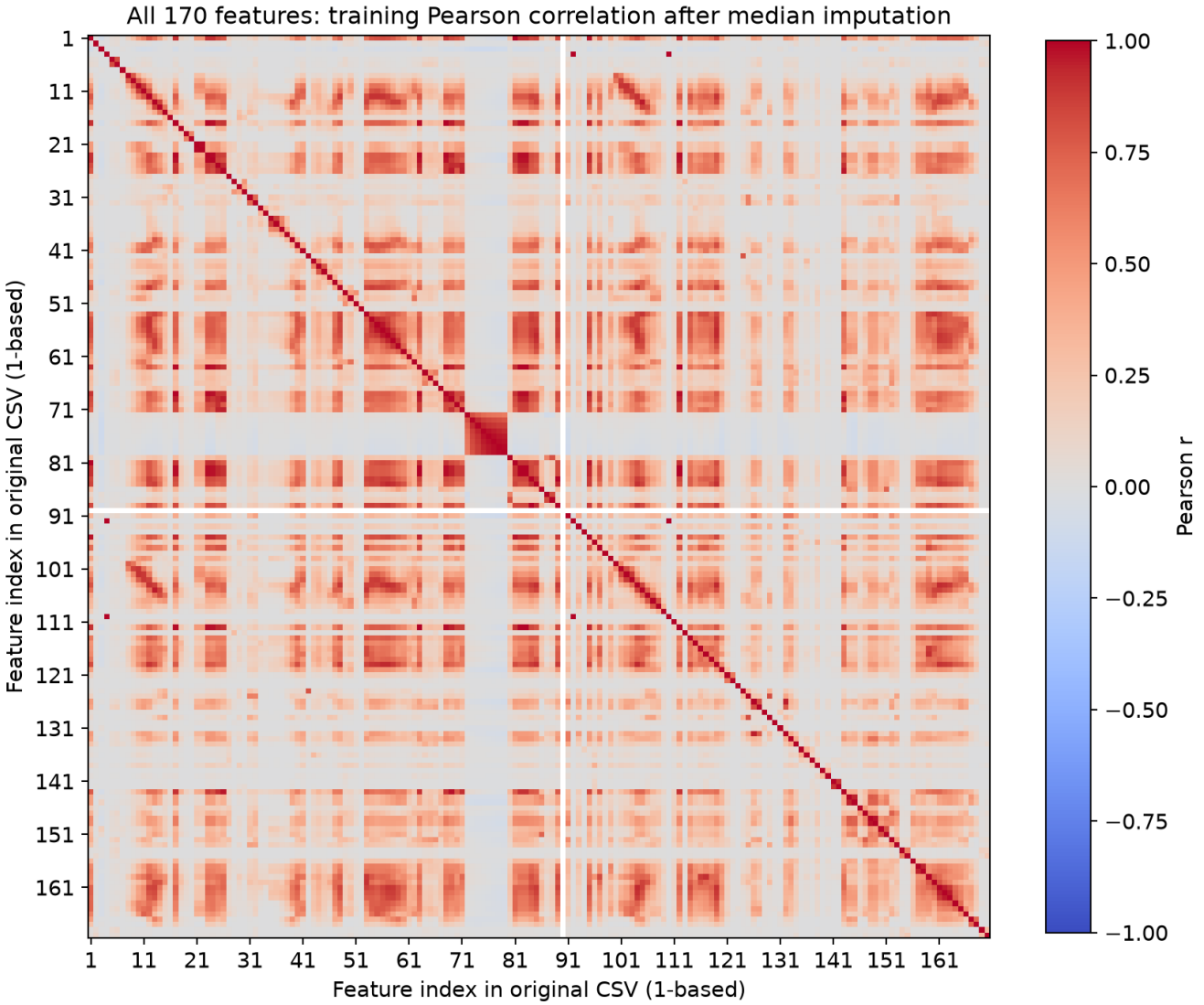
All 170 coefficients are saved in results/feature_cv.csv. There are 0 zero-mean features and 0 entirely missing training features. If a mean were zero, its CV would be undefined (NaN) and excluded from ranking; an entirely missing column would be retained with a zero-fill fallback and flagged. No such fallback is needed here. A zero-variance feature with nonzero mean has CV zero.

Rank	Feature	Training mean	Sample SD	CV
1	cf_000	1.4311e+05	3.5045e+07	244.8875
2	co_000	1.4333e+05	3.5045e+07	244.5074
3	ad_000	1.4344e+05	3.5045e+07	244.3228
4	cs_009	770.51	1.8333e+05	237.9306
5	dh_000	4392.6	5.4124e+05	123.2161
6	dj_000	37.271	4379.2	117.4942
7	ag_000	219.16	20364	92.9178
8	as_000	125.41	10952	87.3325
9	ay_009	1149.6	97412	84.7337
10	ak_000	907.66	72999	80.4250
11	az_009	41.603	3238.3	77.8385
12	ch_000	0.00033333	0.025818	77.4539
13	au_000	228.16	15717	68.8828

The 13 selected features are floor(sqrt(170)) = 13. Large CV values indicate dispersion relative to a possibly tiny mean; rare very large counter values can dominate this ranking. A high CV is not a measure of predictive importance or statistical significance.

These 13 predictors are selected exclusively for visualization. Every random forest and XGBoost fit uses all 170 features. No model uses this ranking for feature selection, and no ranking uses the official test set.

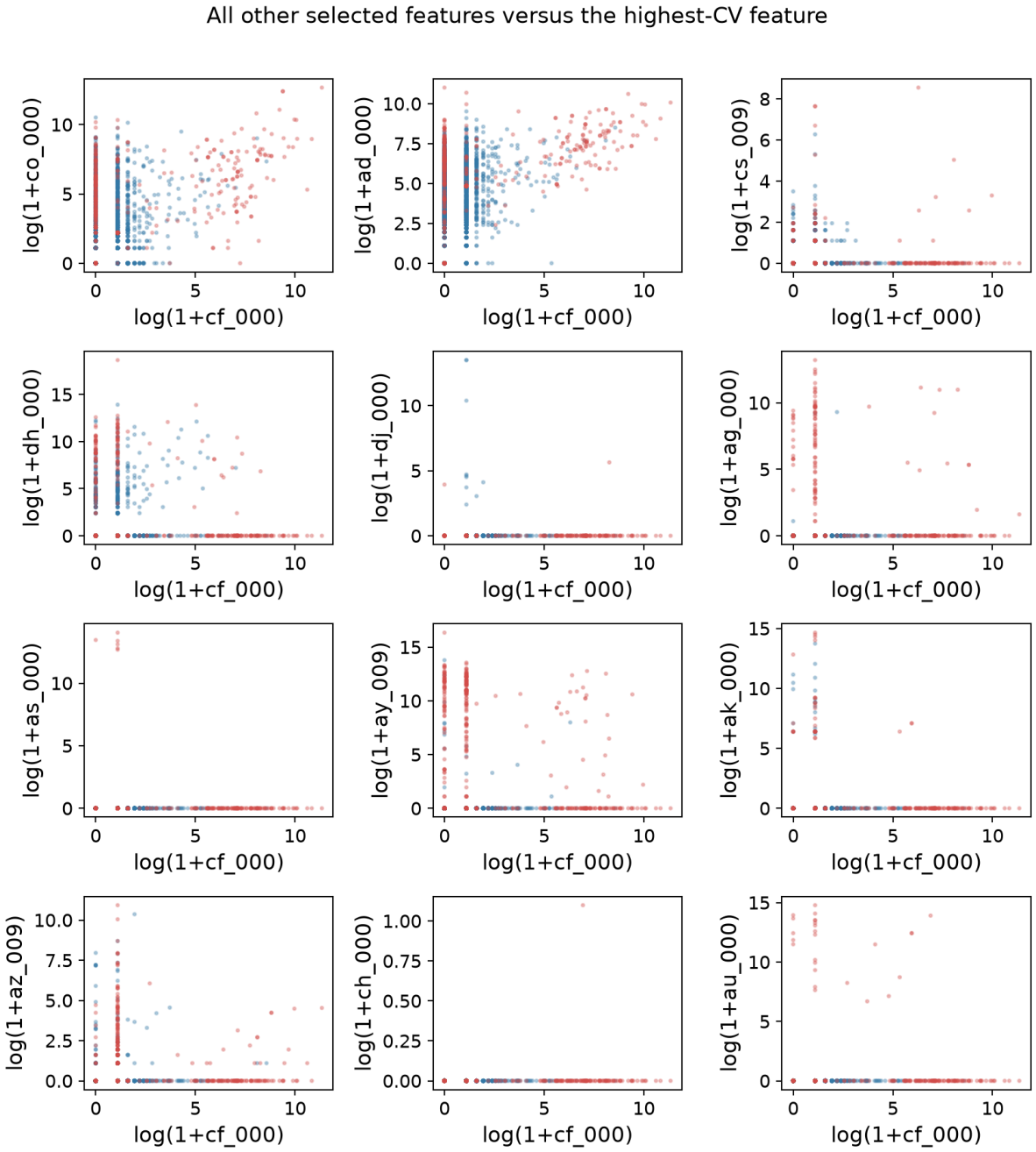
1(b)(iii). Correlation of all 170 features



Pearson correlations use all imputed training observations. Axes retain the CSV feature order and use 1-based indices to keep the full matrix readable on paper. The exact index-to-name mapping is in results/feature_index.csv; the large, individually labeled version is figures/correlation_all_labels.pdf (also PNG). The complete numerical matrix is results/correlation.csv.

The blocks of strong positive association are consistent with related operational counters and histogram bins. Correlation describes marginal linear relationships; it neither proves causation nor captures all nonlinear interactions. Median imputation can change correlations. The diagonal is one, and the numerical matrix is checked for symmetry. Strong correlations can make separate feature effects difficult to interpret.

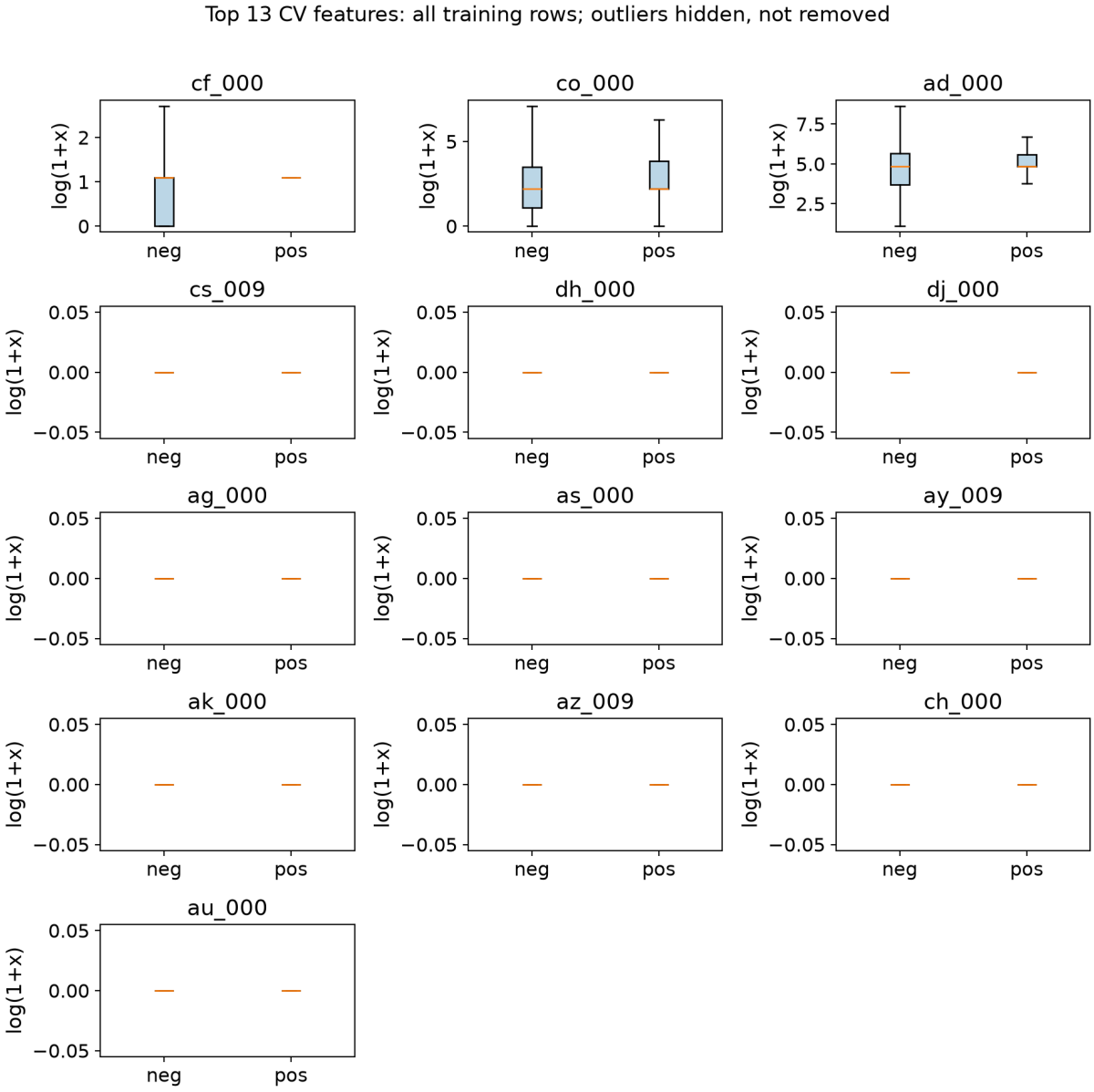
1(b)(iv). Scatter plots for the top 13 CV features



Each of the other 12 selected features is plotted against the highest-CV feature. Blue denotes neg and red denotes pos. For readable skewed counters, axes use log(1+x). Sampling uses seed 552 and includes all 1,000 positives plus 2,500 randomly selected negatives: 3,500 training observations. This deliberate class-stratified display is not representative of class prevalence.

The complete 13-by-13 scatter matrix, including every pair, is provided in the executed notebook and figures/scatter_all13.pdf. No model is fitted to the plotting subsample. The transformations are for visualization only.

1(b)(iv)-(v). Box plots and class imbalance



Boxes use all 60,000 imputed training rows, separated by class, with log(1+x) axes. Outlier markers are hidden to expose the boxes; no observations are deleted. Several low-frequency features concentrate at zero, so a class box can collapse to a line.

Scatter plots alone cannot establish significance: they omit conditional effects, have overplotting, and involve many inspected pairs. A visual association does not supply a hypothesis test or independent evidence of predictive value. Model-based validation or a carefully specified inferential analysis would be needed.

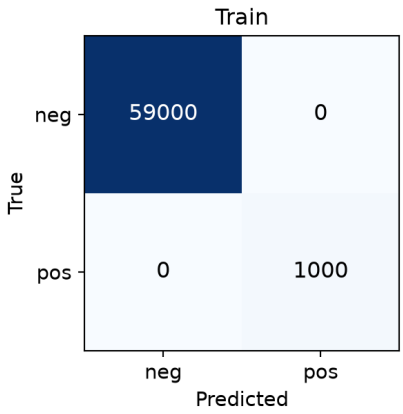
The training ratio is 59:1 (positive prevalence 1.667%); test prevalence is 2.344% with 15,625 negatives and 375 positives. Both splits are severely imbalanced. Always predicting neg yields errors of 1.667% and 2.344%, respectively, while missing every positive. Therefore positive precision, recall, F1 and average precision accompany the required error and ROC-AUC.

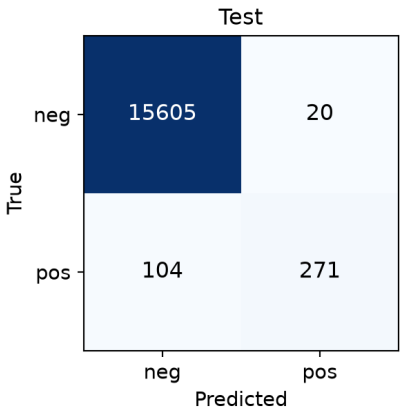
1(c). Random forest without imbalance compensation

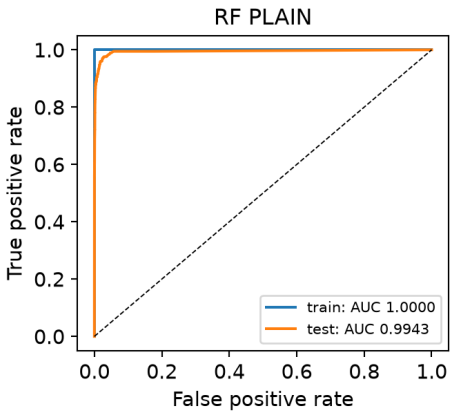
Both forests use 500 fully grown Gini trees, bootstrap sampling, sqrt(170) candidate features per split, minimum leaf size 1, seed 552 and 4 workers. The entire official training set is used. A prediction is pos when the positive-class probability is at least 0.5 (including exact ties). ROC and AUC use probabilities, not hard labels.

This baseline uses class_weight=None and no resampling. OOB predictions aggregate only trees for which the observation was absent from the bootstrap sample. The reported OOB error uses the same 0.5 rule as all other confusion matrices.

Split	Error	ROC-AUC	AP	Precision	Recall	F1
Train	0.00000	1.00000	1.00000	1.00000	1.00000	1.00000
Test	0.00775	0.99435	0.92609	0.93127	0.72267	0.81381







OOB error = 0.00582; official test error = 0.00775; test minus OOB = 0.00193. Fit and evaluation took 70.9 seconds. OOB uses the global training imputation, so it is an approximate internal estimate rather than a fully cross-fitted preprocessing estimate. Test prevalence also differs from training.

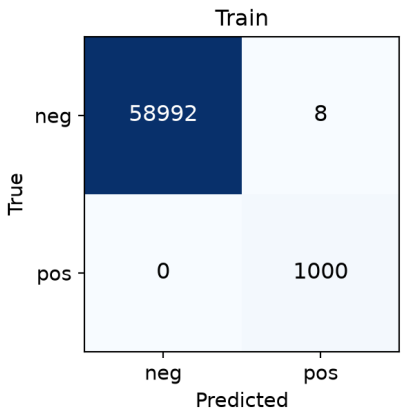
Confusion matrices have true labels in rows and predicted labels in columns, ordered [neg, pos]. Training performance is resubstitution performance and can be optimistic for deep forests.

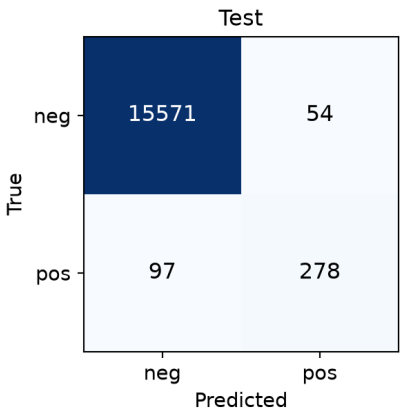
1(d). Random forest with class weighting

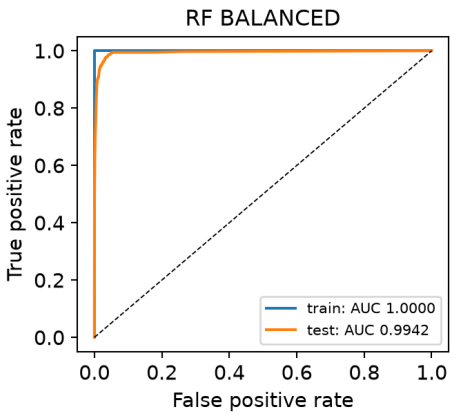
Both forests use 500 fully grown Gini trees, bootstrap sampling, sqrt(170) candidate features per split, minimum leaf size 1, seed 552 and 4 workers. The entire official training set is used. A prediction is pos when the positive-class probability is at least 0.5 (including exact ties). ROC and AUC use probabilities, not hard labels.

Chen, Liaw and Breiman [3] study cost-sensitive weighting and sampling for imbalanced forests. Here class_weight="balanced" weights class k by n/(2 n_k): approximately 0.5085 for neg and 30 for pos, a 59-fold ratio. All other forest settings are unchanged. This weights split impurity and fitted class probabilities; it does not guarantee higher minority recall [4].

Split	Error	ROC-AUC	AP	Precision	Recall	F1
Train	0.00013	1.00000	1.00000	0.99206	1.00000	0.99602
Test	0.00944	0.99418	0.88701	0.83735	0.74133	0.78642







OOB error = 0.01998; official test error = 0.00944; test minus OOB = -0.01055. Fit and evaluation took 44.3 seconds. OOB uses the global training imputation, so it is an approximate internal estimate rather than a fully cross-fitted preprocessing estimate. Test prevalence also differs from training.

Confusion matrices have true labels in rows and predicted labels in columns, ordered [neg, pos]. Training performance is resubstitution performance and can be optimistic for deep forests.

Compared with the unweighted forest, weighting changes test recall from 0.7227 to 0.7413 and precision from 0.9313 to 0.8373. Its observed behavior, rather than the compensation label alone, determines whether it helps. A small overall error can be achieved by emphasizing the abundant negatives; recall directly exposes missed APS failures.

1(e). XGBoost interpretation and validation design

The installed XGBoost version is 3.4.1. Its local XGBClassifier API and the matching v3.4.1 parameter documentation were inspected [5]. The homework describes logistic regressions at tree nodes, but the standard gbtree booster uses feature-threshold splits and scalar leaf scores. binary:logistic applies a logistic loss/link to the summed score. reg_alpha is an L1 penalty on leaf weights; it does not fit a logistic regression at every node.

$$P(Y=1 \mid x)=\sigma\left(\sum_{t=1}^{T} f_t(x)\right), \quad \Omega(f_t)=\gamma L_t+\frac{\lambda}{2}\sum_{\ell} w_{t\ell}^2+\alpha\sum_{\ell}|w_{t\ell}|$$

gblinear instead fits a global linear model, with L1-regularized coefficients; it is not a tree model. A true logistic model tree combines a tree with local logistic models and needs a different implementation. No local course clarification was found. The runnable interpretation here is gbtree + binary:logistic + tuned reg_alpha; it satisfies the XGBoost/alpha experiment but does not claim to realize the described node-level model tree. Instructor confirmation of that wording remains appropriate.

Fixed before test evaluation: 250 rounds, depth 4, learning rate 0.08, L2 penalty 1, max_bin 256, min_child_weight 1, subsample = colsample_bytree = 1, histogram trees on CPU, 4 threads, base_score 0.5. There is no early stopping or test-based choice of rounds/thresholds. There is no class weighting or resampling in this section.

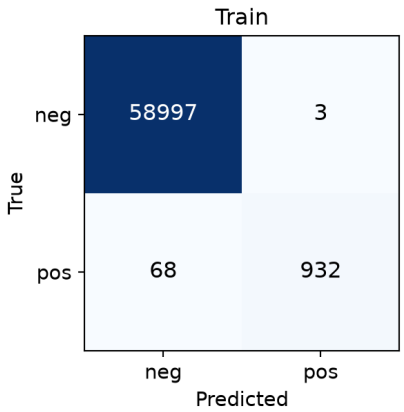
Stratified shuffled 5-fold CV (seed 552) uses the full 60,000 training rows, with 48,000 fitting rows and 12,000 validation rows per fold. The imputer and StandardScaler are fitted inside the training fold. Scaling is shared with 1(f) to isolate the SMOTE comparison; it is not needed for tree splits. Alpha maximizes mean validation average precision (AP), with smaller alpha breaking exact ties. AP rewards ranking of rare positives; CV misclassification at 0.5 is also reported.

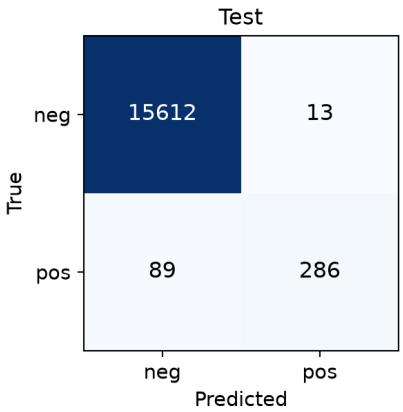
Alpha	Mean CV AP	AP SD	Mean CV error	Error SD
0	0.88760	0.01406	0.00562	0.00055
0.001	0.88568	0.01212	0.00577	0.00054
0.01	0.88727	0.01173	0.00568	0.00064
0.1	0.88643	0.01335	0.00582	0.00064
1	0.88472	0.01300	0.00598	0.00084
10	0.86932	0.01362	0.00658	0.00063
100	0.78346	0.02698	0.00943	0.00089

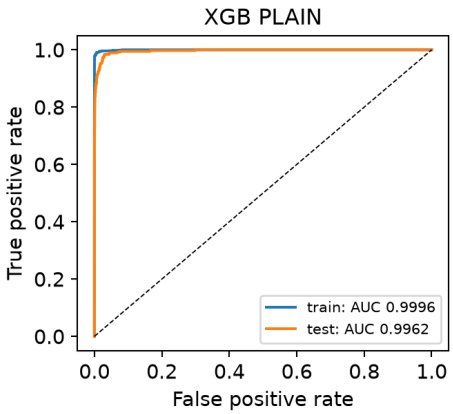
1(e). XGBoost without compensation: results

The selected alpha is 0. Mean validation AP is 0.88760; mean CV misclassification error is 0.00562, with fold SD 0.00055. Selection used only training-fold validation predictions.

Split	Error	ROC-AUC	AP	Precision	Recall	F1
Train	0.00118	0.99959	0.99088	0.99679	0.93200	0.96331
Test	0.00637	0.99623	0.93199	0.95652	0.76267	0.84866







Test error (0.00637) minus selected-alpha CV error (0.00562) is 0.00076. CV fits use 80% of training rows and train-only fold preprocessing, whereas the final model uses all training rows and the main test evaluation follows the footnote. Distribution/prevalence differences can also affect this gap.

The selected mean CV performance is a tuning estimate, not an independent, unbiased estimate of the whole selection procedure. Nested CV was not required or performed. The untouched official test set gives the final evaluation after tuning. Fold SD is descriptive variation across five dependent folds, not a confidence interval.

The final fit and evaluation took 1.9 seconds; the 35 CV fits took 99.4 seconds in total. Full per-fold scores and times are saved in results/xgb_plain_cv_folds.csv. AP is sklearn average_precision_score (a recall-weighted precision sum), not trapezoidal PR-AUC.

1(f). SMOTE: full-data and fold-safe design

SMOTE creates a synthetic positive between an observed minority point and one of its minority nearest neighbors [7]. A single seeded imblearn Pipeline applies: training-fold median imputer, training-fold StandardScaler, SMOTE(k_neighbors=5, sampling_strategy=1.0), then the same XGBoost estimator as 1(e). CV runs serially and each model uses 4 threads to avoid nested parallel oversubscription.

$$x_{new} = x_i + u(x_{neighbor} - x_i), \qquad u \sim U(0, 1)$$

Distance is computed after standardizing all 170 features using current training-fold statistics; otherwise large-unit counters dominate Euclidean neighbors. Standardization still cannot eliminate high-dimensional distance issues, skew or imputation artifacts. No log transformation is applied in the model pipeline.

Stage	neg	Original pos	Final pos	Total
Each CV training fold	47,200	800	47,200	94,400
Final full training fit	59,000	1,000	59,000	118,000
Each validation fold (unchanged)	11,800	200	200	12,000
Official test (unchanged)	15,625	375	375	16,000

The validation rows are never passed to fit_resample; resampling before splitting would leak neighbor information and invalidate validation [8]. Cache reuse is restricted to identical training-fold inputs and settings. Final SMOTE uses the entire original training set without downsampling. All reported training metrics are evaluated on the original 60,000 observations, not the synthetic population.

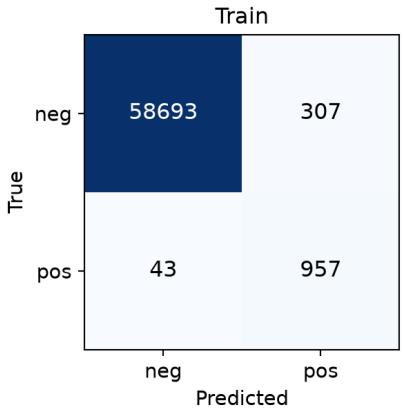
The same folds, alpha grid, selection metric, fixed rounds and classification rule are used as in 1(e). Alpha is reselected for the SMOTE procedure; its effect depends on changed sample size and gradients.

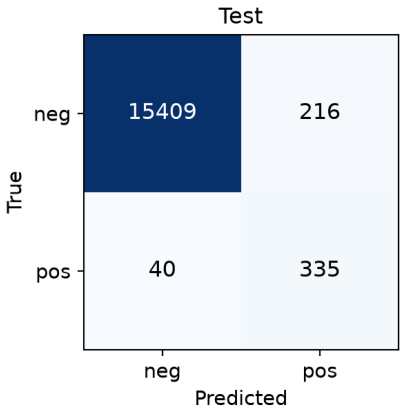
Alpha	Mean CV AP	AP SD	Mean CV error	Error SD
0	0.84750	0.02992	0.00977	0.00057
0.001	0.84761	0.02998	0.00975	0.00057
0.01	0.84911	0.02871	0.00985	0.00065
0.1	0.84890	0.03101	0.00990	0.00093
1	0.84974	0.03083	0.00990	0.00083
10	0.84177	0.02887	0.01052	0.00070
100	0.78665	0.02999	0.02017	0.00159

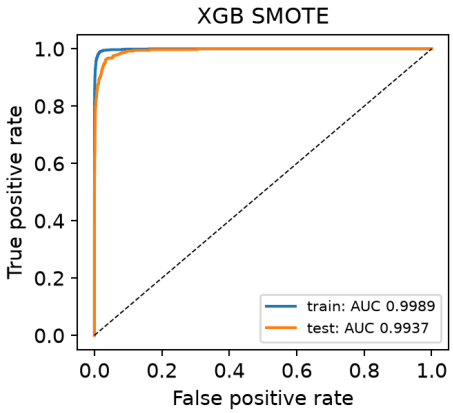
1(f). SMOTE + XGBoost: results and tradeoffs

Selected alpha = 1; mean CV AP = 0.84974; mean CV error = 0.00990 (fold SD 0.00083). The final data contain 59,000 negatives and 59,000 positives. Full-data SMOTE took 0.13 seconds, and final scaling, SMOTE, fit and evaluation took 4.7 seconds.

Split	Error	ROC-AUC	AP	Precision	Recall	F1
Train	0.00583	0.99885	0.96342	0.75712	0.95700	0.84541
Test	0.01600	0.99365	0.88362	0.60799	0.89333	0.72354







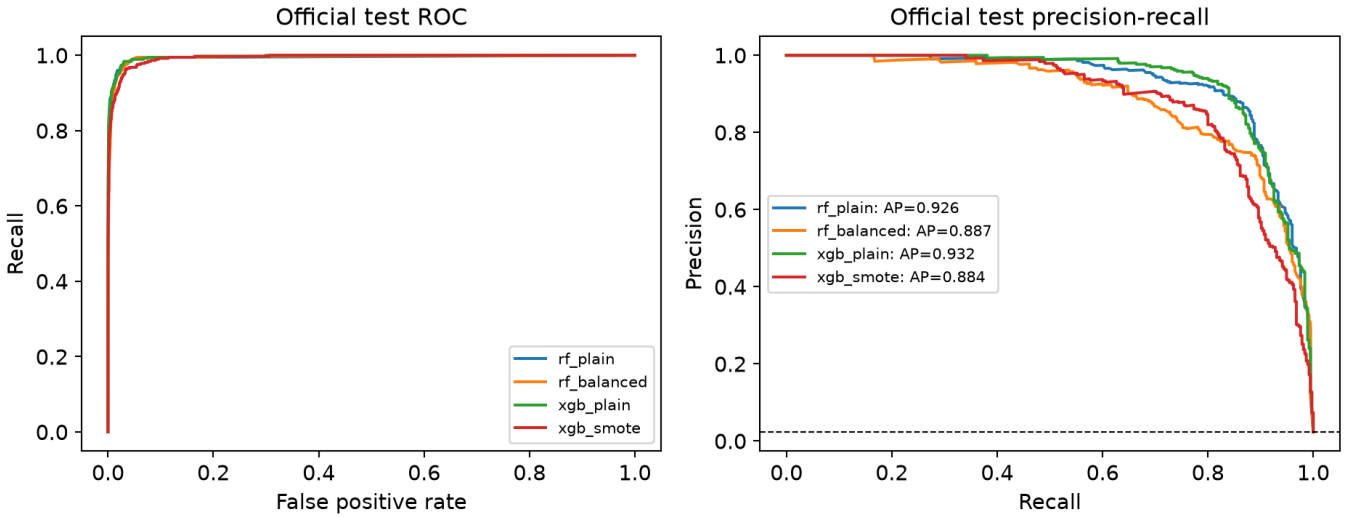
Test error minus selected-alpha CV error is 0.00610. The 35 CV fits required 143.6 seconds in total, including fold preprocessing and cached reuse. Both alpha searches include all original training observations; only the fold fitting sets are oversampled.

Without SMOTE, test precision/recall are 0.9565/0.7627, with 13 false positives and 89 false negatives. With SMOTE they are 0.6080/0.8933, with 216 false positives and 40 false negatives. These counts expose the minority-recall versus unnecessary-inspection tradeoff.

Oversampling changes the fitted class prior and may distort probability calibration. A 0.5 threshold is retained for comparability, rather than optimized on official test labels. Scores should not automatically be interpreted as calibrated failure probabilities in the original population.

1(c)-(f). Unified comparison and sensitivity

Model / test	Error	ROC-AUC	AP	Prec.	Recall	F1
RF: unweighted	0.0077	0.9943	0.9261	0.9313	0.7227	0.8138
RF: balanced	0.0094	0.9942	0.8870	0.8373	0.7413	0.7864
XGB: unweighted	0.0064	0.9962	0.9320	0.9565	0.7627	0.8487
XGB: SMOTE	0.0160	0.9937	0.8836	0.6080	0.8933	0.7235



Sensitivity: fit the imputer on training data only

Each model is held fixed; only missing official-test values change from test medians (main results) to fitted training medians. The scaler, where present, always uses training statistics. This is a post-fit robustness comparison, not an additional selection criterion. SMOTE is sensitive to this convention: train-only test imputation reduces its error from 1.600% to 1.075%, with false positives falling from 216 to 125 and false negatives rising from 40 to 47. Synthetic neighbors can amplify shifts in imputed feature values. This makes the preprocessing convention a material part of the result.

Model	Main error	Train-only error	Main AP	Train-only AP
RF: unweighted	0.00775	0.00775	0.92609	0.92614
RF: balanced	0.00944	0.00944	0.88701	0.88716
XGB: unweighted	0.00637	0.00625	0.93199	0.93160
XGB: SMOTE	0.01600	0.01075	0.88362	0.89974

ROC-AUC measures ranking across both classes; AP focuses on precision along minority recall and depends on prevalence. The dashed PR baseline is the test positive prevalence (0.02344). No model or operating threshold was selected using this figure. Comparisons apply to this fixed protocol and one official test set.

The data description assigns cost 10 to an unnecessary inspection and 500 to a missed failure. For context, test cost (10 FP + 500 FN) is RF: unweighted: 52,200, RF: balanced: 49,040, XGB: unweighted: 44,630, XGB: SMOTE: 22,160. These retrospective costs did not guide parameter or threshold selection.

Question 2. ISLR 6.6.3: relaxing the lasso constraint

Edition check: ISLR second edition (2021, corrected June 21, 2023), Section 6.6, Exercise 3. The first edition places the same exercise in 6.8. The problem minimizes training residual sum of squares with an unpenalized intercept under an L1 coefficient bound s [1].

$$\min_{\beta_0,\beta} \sum_i \left(y_i - \beta_0 - \sum_j x_{ij}\beta_j \right)^2 \quad \text{subject to} \quad \sum_j |\beta_j| \le s$$

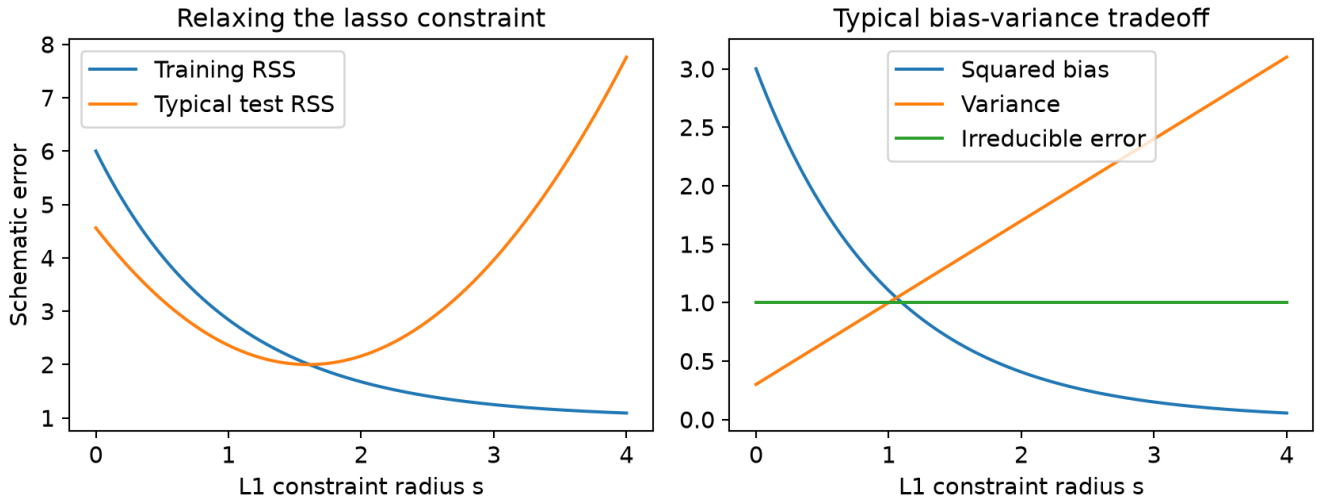
(a) Training RSS: option iv, decreases (more precisely, is nonincreasing). If s2 is at least s1, every solution feasible at s1 remains feasible at s2. Minimizing over a larger set cannot increase training RSS. At s = 0, all slopes vanish and only the sample-mean intercept remains. Once a least-squares solution is feasible, RSS can stay constant.

(b) Test RSS: option ii, typically decreases and then increases (a U shape). Initially loosening a tight constraint can reduce underfitting. Excess flexibility eventually increases estimation variance and can worsen held-out fit. This is the intended bias-variance pattern, not a guaranteed shape for every realized test set.

(c) Variance: option iii, typically increases. A larger radius weakens shrinkage and allows coefficients to react more strongly to training-sample noise.

(d) Squared bias: option iv, typically decreases. Less shrinkage removes the distortion imposed by a tight L1 bound. This describes the usual statistical trend rather than a universal monotonic theorem for arbitrary designs and data-generating functions.

(e) Irreducible error: option v, remains constant. The conditional noise variance is a property of the data-generating process; changing a fitting constraint cannot change it.



These plots are schematic illustrations, not outputs of the APS experiments. Only the nested-feasible-set argument in (a) guarantees monotonicity; actual test curves may be noisy or have their minimum at a boundary.

Question 3. ISLR 6.6.5: identical predictors

ISLR second edition Section 6.6 Exercise 5 (first edition: 6.8.5). Let x1 = x11 = x12 and x2 = x21 = x22, with centered x and y, so the fitted intercept is zero. Assume a nonzero predictor column and a positive penalty unless stated otherwise. Put t = beta1 + beta2, A = x1 squared + x2 squared > 0, and C = x1 y1 + x2 y2.

(a) Ridge objective

$$\min_{\beta_1,\beta_2} (y_1 - x_1\beta_1 - x_1\beta_2)^2 + (y_2 - x_2\beta_1 - x_2\beta_2)^2 + \lambda(\beta_1^2 + \beta_2^2)$$

(b) Why the ridge coefficients are equal

For a fixed sum t, the residual term depends only on t. The identity below shows that the penalty is uniquely minimized by beta1 = beta2 = t/2. Equivalently, subtracting the two normal equations gives 2 lambda (beta1 - beta2) = 0. Strict convexity from lambda > 0 makes the solution unique.

$$\beta_1^2 + \beta_2^2 = \frac{t^2}{2} + \frac{(\beta_1 - \beta_2)^2}{2}, \qquad \hat{\beta}_1 = \hat{\beta}_2 = \frac{C}{2A + \lambda}$$

(c) Lasso objective

$$\min_{\beta_1,\beta_2} (y_1 - x_1\beta_1 - x_1\beta_2)^2 + (y_2 - x_2\beta_1 - x_2\beta_2)^2 + \lambda(|\beta_1| + |\beta_2|)$$

(d) Describe all minimizing coefficient pairs

The triangle inequality gives |beta1| + |beta2| >= |t|, with equality whenever the two coefficients have the same sign (allowing zero). Thus minimizing reduces to the one-dimensional objective A t squared - 2 C t + lambda |t| plus a constant. Its minimizer is the soft-thresholded sum below.

$$t^* = \frac{\text{sign}(C)(|C| - \lambda/2)_+}{A}, \qquad (\hat{\beta}_1, \hat{\beta}_2) = (at^*, (1-a)t^*), \quad 0 \le a \le 1$$

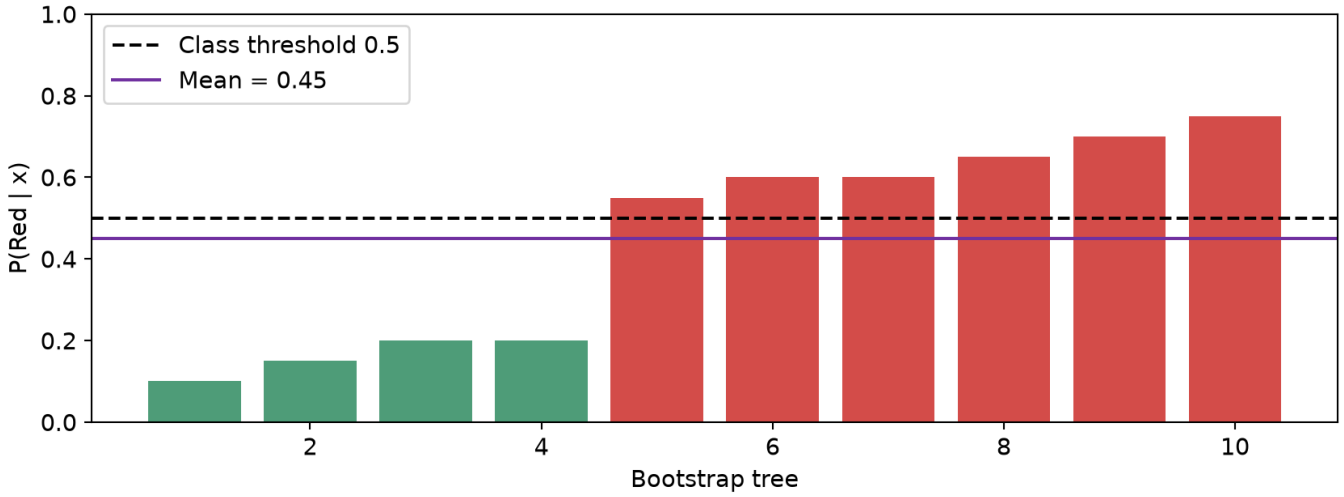
When t* is nonzero, this continuum includes selecting only the first predictor, selecting only the second, and every same-sign split between them. All pairs have identical fitted values and identical L1 penalty. This explains why a lasso solver can treat perfectly correlated variables differently.

Boundary qualifications: if lambda >= 2|C| and lambda > 0, t* = 0 and only (0,0) minimizes, so nonuniqueness is not universal. If lambda = 0, any pair summing to C/A is least-squares optimal, including opposite signs; the equality claim for ridge is then not forced. These qualifications refine the exercise's generic nonuniqueness statement.

Question 4. ISLR 8.4.5: votes versus probabilities

Both checked R editions place this exercise in Section 8.4, Exercise 5. For one fixed predictor value, ten bootstrap classification trees return the following red-class probabilities [1].

Tree	1	2	3	4	5	6	7	8	9	10
P(Red)	0.10	0.15	0.20	0.20	0.55	0.60	0.60	0.65	0.70	0.75



Majority vote: Red

Applying a 0.5 rule separately to each tree gives four Green votes from trees 1-4 and six Red votes from trees 5-10. Six of ten hard predictions are Red, so the majority-vote ensemble predicts Red. No individual probability is exactly 0.5, so the tie convention is irrelevant.

Average probability: Green

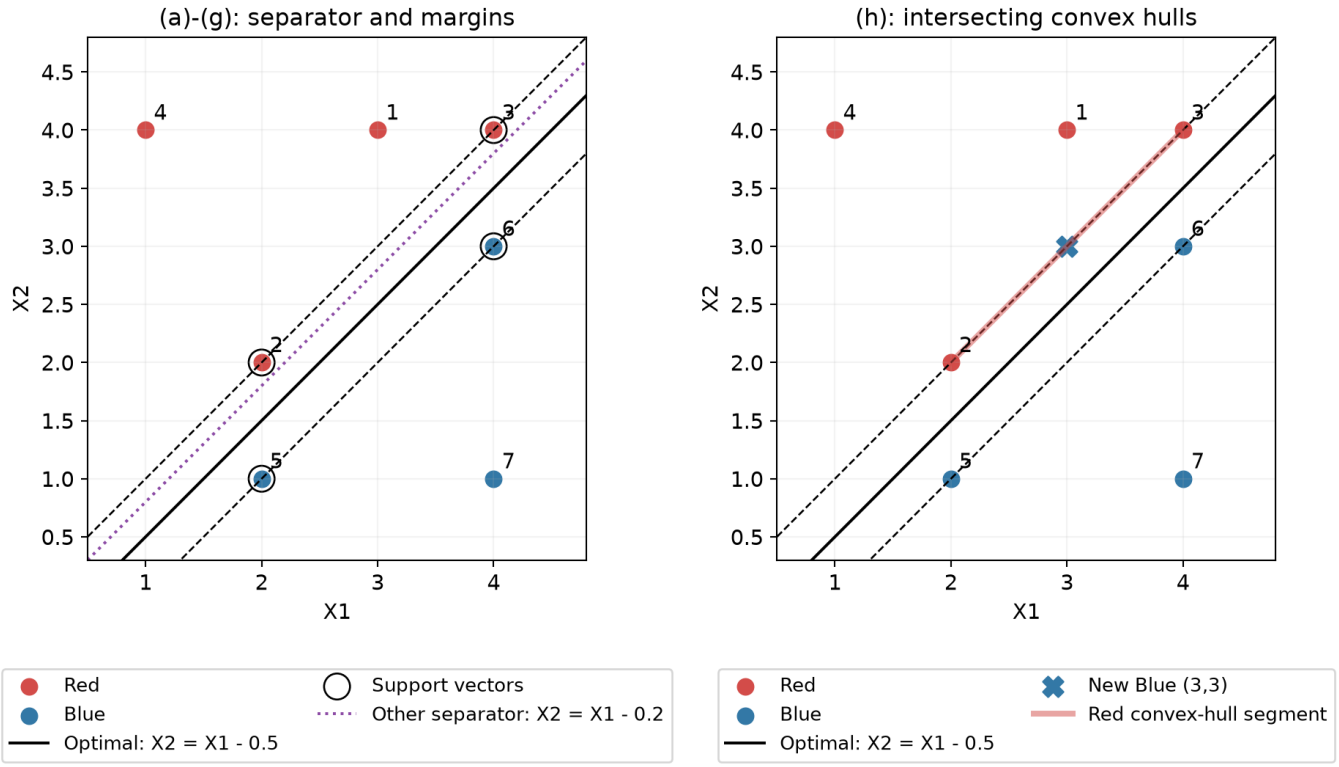
$$\bar{p}_{Red} = \frac{0.10 + 0.15 + 0.20 + 0.20 + 0.55 + 0.60 + 0.60 + 0.65 + 0.70 + 0.75}{10} = 0.45$$

Averaging probabilities gives Red probability 0.45 and Green probability 0.55, so the probability-average rule predicts Green. The few low Red probabilities can outweigh a larger number of moderately high Red probabilities.

Hard voting counts how many models favor a class and discards their confidence. Probability averaging retains the numerical magnitudes. The two aggregation methods need not agree. The random forests in Question 1 use sklearn probability averaging followed by the explicitly recorded 0.5 threshold; this exercise should not be read as a claim that every software implementation votes on hard labels.

Question 5. ISLR 9.7.3: maximal-margin geometry

Both checked R editions place this seven-observation problem in Section 9.7, Exercise 3. The numbered observations and constructions are shown below; the black dashed lines are supporting lines, not additional decision boundaries.



(a) Red observations are 1=(3,4), 2=(2,2), 3=(4,4), 4=(1,4); Blue observations are 5=(2,1), 6=(4,3), 7=(4,1).

(b) The optimal boundary is X2 = X1 - 0.5, or 0.5 - X1 + X2 = 0. The nearest Red and Blue points lie on X2 = X1 and X2 = X1 - 1. Their convex hulls are 1/sqrt(2) apart; the midpoint line attains half this distance, the largest possible margin.

(c) Classify Red if 0.5 - X1 + X2 > 0; otherwise classify Blue. Thus (beta0, beta1, beta2) = (0.5, -1, 1); a positive rescaling gives the same classifier.

(d) The geometric margin on each side is 0.5/sqrt(2) = 0.353553. The full width between supporting lines is 1/sqrt(2) = 0.707107. In canonical functional-margin units use (1, -2, 2), with support scores +1 or -1.

(e) Support vectors are observations 2, 3, 5 and 6, on the dashed supporting lines. An independent hard-margin SVM approximation verifies these points, coefficients (-2,2), and intercept 1.

(f) Observation 7 has score 0.5 - 4 + 1 = -2.5. A sufficiently small movement keeps it strictly beyond the Blue supporting line, leaving the active margin constraints and optimal boundary unchanged.

(g) The purple dotted separator X2 = X1 - 0.2 still separates every original point. Its closest distance is 0.2/sqrt(2), smaller than 0.5/sqrt(2), so it is not optimal.

(h) Add a Blue point at (3,3), the midpoint of Red observations 2=(2,2) and 3=(4,4). A linear functional positive at both Red endpoints is positive at their midpoint. The convex hulls intersect, so no hyperplane can strictly separate the classes.

References, assumptions and reproducibility

[1] James, G., Witten, D., Hastie, T., and Tibshirani, R. An Introduction to Statistical Learning, with Applications in R, 2nd ed. (2021), corrected June 21, 2023; compared against first-edition seventh printing (2013). Author-provided downloads: https://www.statlearning.com/ . Second-edition download redirect: https://hastie.su.domains/ISLR2/ISLRv2_corrected_June_2023.pdf.download.html . Verified exercise locations and source-page images are retained locally.

[2] Scikit-learn, Imputation of missing values: https://scikit-learn.org/stable/modules/impute.html .

[3] Chen, C., Liaw, A., and Breiman, L. (2004). Using Random Forest to Learn Imbalanced Data. Technical Report 666, UC Berkeley. https://statistics.berkeley.edu/sites/default/files/tech-reports/666.pdf .

[4] Scikit-learn, RandomForestClassifier API (class_weight and probability prediction): https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html .

[5] XGBoost v3.4.1 parameter source: https://github.com/dmlc/xgboost/blob/v3.4.1/doc/parameter.rst ; Python API: https://xgboost.readthedocs.io/en/stable/python/python_api.html .

[6] XGBoost, Introduction to Boosted Trees: https://xgboost.readthedocs.io/en/stable/tutorials/model.html .

[7] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR 16, 321-357. https://doi.org/10.1613/jair.953 .

[8] Imbalanced-learn, Common pitfalls and recommended practices: https://imbalanced-learn.org/stable/common_pitfalls.html .

[9] Scania CV AB (2016). APS Failure and Operational Data for Scania Trucks, supplied data description and official train/test CSVs. Dataset record: https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks .

Reproducibility and scope

Python 3.12.8; numpy 2.5.4, pandas 3.1.1, scikit-learn 1.9.1, xgboost 3.4.1, imbalanced-learn 0.14.2, matplotlib 3.11.2, nbformat 5.11.1, nbclient 0.11.0, reportlab 5.1.0. CPU-only execution on an 8-logical-CPU, 16 GiB machine. No GPU assumed. Five folds run sequentially, estimator threads = 4. Exact pins and saved environment metadata accompany this submission.

The notebook contains code, saved-result checks, all formal answers and rendered figures. scripts/experiments.py can resume completed stages; deleting only the documented generated result files triggers fresh fits. Source data are never rewritten. The 70 fold evaluations, final predictions and models, metrics, hyperparameters, timings, and fold memberships are retained.

Question 6 is explicitly Extra Practice and is recorded separately in CHECKLIST.md, excluded from this submitted report. The remaining interpretation issue is whether the instructor intended a genuine logistic model tree rather than the standard XGBoost implementation documented in 1(e). There are no unresolved formal exercise statements.